# MOBL-07: Network Request Monitoring

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 7 of 12 | **Created:** February 2026 | **Last Updated:** 10/29/2026

## Overview

Every mobile app depends on the network. Whether it is fetching user profiles, loading product catalogs, or submitting orders, HTTP(S) requests are the lifeline between a mobile frontend and its backend services. Dynatrace automatically captures these network requests from instrumented mobile apps, providing deep visibility into:

- **What** is being requested (URL, method, status code)
- **How long** each request takes ( `duration` )
- **How much** data is transferred (request and response sizes)
- **What connection** the device is using (WiFi, 5G, LTE, 3G)
- **Where** the request goes on the backend (distributed trace correlation)

This notebook explains how Dynatrace captures network requests across platforms, how to query and analyze them with DQL, and how to identify slow or failed requests that degrade the mobile user experience.

---

## Table of Contents


1. Automatic HTTP Capture
2. Connection Type Detection
3. Request Timing Breakdown
4. Frontend-to-Backend Correlation
5. Querying Network Requests

6. Slow & Failed Requests
7. Performance Optimization Tips

---

# Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with Grail |
| **Mobile App Instrumented** | Dynatrace Mobile SDK deployed (iOS, Android, Flutter, or React Native) |
| **Network Requests** | Mobile app actively making HTTP(S) requests being captured by the SDK |
| **Permissions** | `storage:user.events:read`, `storage:spans:read` |
| **Distributed Tracing** | Backend services instrumented with OneAgent, OpenTelemetry, or both for correlation |

# 1. Automatic HTTP Capture

The Dynatrace Mobile SDK automatically intercepts HTTP(S) requests made through standard networking libraries on each platform. No manual instrumentation is required for the libraries listed below.

## Supported Platforms and Libraries

| Platform | HTTP Library | Auto-Captured |
|---|---|---|
| iOS | URLSession | Yes |
| iOS | Alamofire (URLSession-based) | Yes |
| Android | OkHttp | Yes |
| Android | HttpURLConnection | Yes |

| Platform | HTTP Library | Auto-Captured |
|---|---|---|
| Flutter | Dart `http` package via `Dynatrace().createHttpClient()` | Yes (wrapper client) |
| Flutter | `dart:io HttpClient` , gRPC, custom stacks | No -- manual web request monitoring |
| React Native | Fetch API, XMLHttpRequest | Yes |

> **Note:** Third-party libraries that use non-standard HTTP stacks (e.g., custom socket implementations) may require manual instrumentation. Check the Dynatrace documentation for your specific SDK version.

## Data Captured Per Request

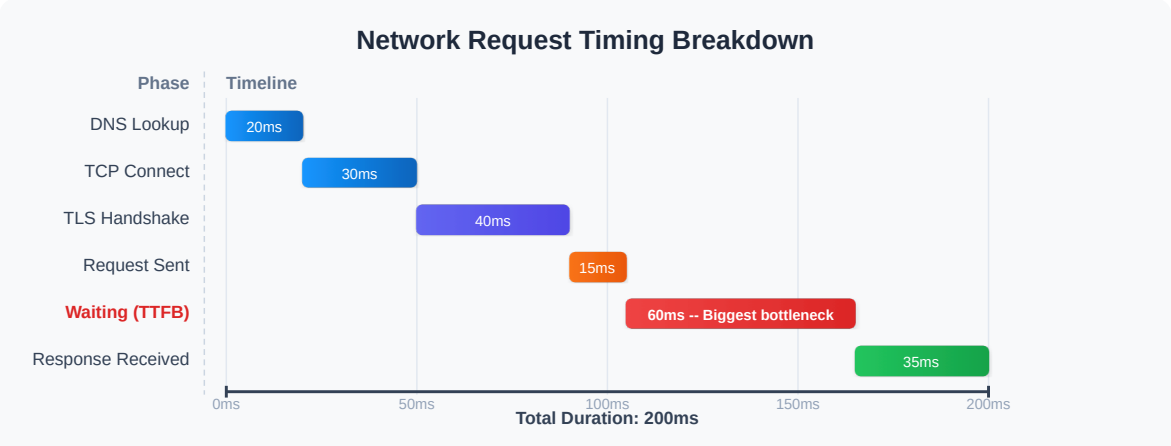
For each intercepted network request, the SDK captures:

| Data Point | Description |
|---|---|
| **URL** | Full request URL (query parameters may be redacted based on privacy settings) |
| **HTTP Method** | GET, POST, PUT, DELETE, PATCH, etc. |
| **Status Code** | HTTP response status code (200, 404, 500, etc.) |
| **Request / Response Size** | Body sizes in bytes ( `http.request.body.size` , `http.response.body.size` ) where the agent reports them |
| **Duration** | Total time from request initiation to response completion |
| **Connection Type** | Network connection type at the time of the request (WiFi, LTE, 5G, etc.) |

The SDK batches captured data and sends it to the Dynatrace cluster at regular intervals, minimizing the performance impact on the mobile app.

# 2. Connection Type Detection

Understanding the network conditions under which requests are made is critical for diagnosing performance issues. A request that takes 3 seconds over a 2G connection is expected, but the same latency over WiFi signals a backend problem.




A diagnostic model of request phases. Mobile request events record the total duration, not each phase.

## Connection Types

The SDK records the connection type at the time of each request in `network.connection.type` (dictionary examples `wifi`, `cell`) and the technology in `network.connection.subtype` (examples `lte`, `802.11x`); the cellular generations below are subtype values. Run `summarize count(), by:{network.connection.type, network.connection.subtype}` on your own data to see the exact values your agents report:

| Connection | Description |
|---|---|
| **WiFi** | Connected via WiFi network |
| **5G** | 5th generation cellular network |
| **LTE/4G** | 4th generation cellular network |
| **3G** | 3rd generation cellular network |
| **2G** | 2nd generation cellular network |

Connection type is stored alongside each network request event, enabling you to:

- **Segment performance by connection type** to set realistic SLOs

- **Identify users on poor connections** who experience degraded performance
- **Build adaptive behavior** in your app based on detected connection quality

## 3. Request Timing Breakdown

A single network request goes through multiple phases, each of which can contribute to perceived latency. This section is a **diagnostic framework**, not a list of captured fields: a mobile request event reports the overall `duration`, status and sizes. The phase-level timings in the `rum_request` model come from the browser's W3C Resource Timing API (`characteristics.has_w3c_resource_timings`) and apply to web frontends. For a slow mobile request, compare `duration` across connection types (Section 2) and follow the backend trace (Section 4) to separate network time from server time.

> **Dictionary:** model `rum_request` (`user.events`) lists `characteristics.has_w3c_resource_timings`, `characteristics.has_w3c_navigation_timings`, `http.request.method`, `http.response.status_code` — no DNS/TCP/TLS phase fields; `network.connection.type` (`experimental`); `http.response.body.size` (`stable`), read 10/02/2026.

### Timing Phases

| Phase | What Happens | What a Slow Phase Indicates |
|---|---|---|
| **DNS Lookup** | Resolves the domain name to an IP address | DNS server issues, missing DNS cache, poor network conditions |
| **TCP Connect** | Establishes a TCP connection to the server | Network latency, server unreachable, firewall delays |
| **TLS Handshake** | Negotiates the TLS/SSL secure connection | Certificate chain issues, slow TLS negotiation, outdated cipher suites |
| **Request Sent** | Transmits the HTTP request body to the server | Large request payload, low upload bandwidth |
| **Waiting (TTFB)** | Time to First Byte -- server processes the request | Slow backend logic, database queries, cold starts, overloaded services |

| Phase | What Happens | What a Slow Phase Indicates |
|---|---|---|
| **Response Received** | Downloads the HTTP response body | Large response payload, low download bandwidth, connection throttling |

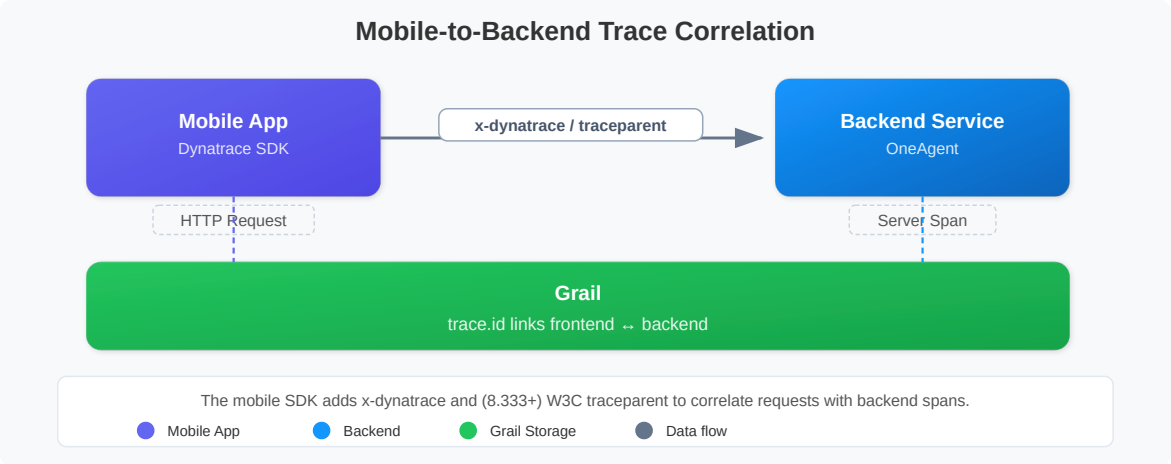
### Diagnostic Guide

| Symptom | Likely Cause | Investigation |
|---|---|---|
| High DNS time | DNS resolution issues | Check DNS provider, enable DNS caching |
| High TCP Connect time | Network path issues | Check CDN proximity, network hops |
| High TLS time | Certificate or cipher issues | Verify certificate chain, enable TLS session resumption |
| High TTFB | Backend performance | Check backend traces, database queries, cold starts |
| High Response time | Large payloads | Enable compression, paginate responses, reduce payload size |

> **Tip:** When the backend span accounts for most of the request's `duration`, the problem is on the backend. Use the frontend-to-backend correlation (Section 4) to trace the request into your server-side services.

## 4. Frontend-to-Backend Correlation

One of the most powerful features of Dynatrace mobile monitoring is the ability to correlate a network request on the mobile device with the corresponding backend service trace. This creates end-to-end visibility from the user's finger tap to the database query.




## How It Works

1. **Request Initiation** -- The mobile app makes an HTTP(S) request using a standard networking library.
2. **Header Injection** -- The Dynatrace Mobile SDK adds the `x-dynatrace` header to the outgoing request (RUM Classic), and from OneAgent for Mobile 8.333 also injects the W3C Trace Context headers `traceparent` and `tracestate`. Custom networking stacks must add the headers manually (for `x-dynatrace`, via `Dynatrace.getRequestTagHeader()` and a request tag -- see MOBL-12 §4).
3. **Backend Processing** -- The backend service (instrumented with OneAgent, OpenTelemetry, or both) reads the trace context and creates a server-side span that is linked to the mobile request.
4. **Trace Stitching** -- Dynatrace automatically stitches the mobile user action and the backend service call into a unified distributed trace.

## What This Enables

| Capability | Description |
|---|---|
| **End-to-end latency** | See total time from mobile request to backend response |
| **Backend root cause** | When TTFB is high, drill into the exact backend service, method, and database call |
| **Service dependency mapping** | Understand which backend services support which mobile features |
| **Error correlation** | Link a mobile HTTP 500 error to the specific backend exception |

> **Important:** For correlation to work, the backend service must be instrumented with Dynatrace OneAgent, OpenTelemetry, or both. W3C trace context works with either.
>
> **Correction (09/28/2026).** Earlier revisions named the mobile correlation header `x-dtc`. That is the web RUM JavaScript header; the mobile SDK uses `x-dynatrace` and, from 8.333, W3C trace context. OneAgent for Mobile 8.333 was released 02/23/2026 with rollout from 02/24/2026; mobile agent versions reach users with app releases, so older builds in the field still send only `x-dynatrace`.
>
> **Sources:** OneAgent SDK for Android (DT docs) — *"To track web requests, add the x-dynatrace HTTP header with a unique value to the web request."*, What's new in OneAgent for Mobile 8.333 (DT docs) — *"OneAgent for Mobile now injects W3C Trace Context headers ( traceparent and tracestate ) into outbound requests"*, Frontend-backend linking (DT docs).

## 5. Querying Network Requests

Network requests from mobile apps are stored in the Grail `user.events` store (`characteristics.has_request`), with `url.full`, `http.request.method`, `http.response.status_code`, `duration` and `network.connection.type`. The following queries demonstrate how to retrieve, aggregate, and analyze mobile network request data.

### Recent Network Requests

Retrieve the most recent network requests from mobile applications, showing the key attributes for each request.

```
// Recent network requests from mobile apps
fetch user.events, from:-1h
| filter dt.rum.application.type == "mobile" and characteristics.has_request
| fields start_time, frontend.name, url.full, http.request.method,
http.response.status_code, duration, network.connection.type
| sort start_time desc
| limit 50
```

### Network Request Volume by Application

Understand which mobile applications are generating the most network traffic. This helps identify apps that may need optimization or capacity planning.

```
// Network request volume by application
fetch user.events, from:-1h
| filter dt.rum.application.type == "mobile" and characteristics.has_request
| summarize request_count = count(), by:{frontend.name}
| sort request_count desc
| limit 20
```

### Trace Coverage of Mobile Requests

Measure how many mobile requests were linked to a backend distributed trace: a request event that carries `trace.id` was linked. A low rate points at backends that are not instrumented, or at app builds older than 8.333. To see the backend side, collect the `trace.id` values and query `fetch spans | filter in(trace.id, ...)`.

```
// Trace coverage of mobile-originated requests
// A request that carries trace.id was linked to backend distributed tracing
// (OneAgent for Mobile 8.333+ injects W3C traceparent / tracestate headers).
fetch user.events, from:-2h
| filter dt.rum.application.type == "mobile" and characteristics.has_request
| summarize {total_requests = count(), traced_requests =
countIf(isNotNull(trace.id))}, by:{frontend.name}
| fieldsAdd trace_rate = 100.0 * traced_requests / total_requests
| sort trace_rate asc
```

## 6. Slow & Failed Requests

Monitoring network request performance over time helps you detect regressions, correlate with deployments, and identify patterns. The following queries create time-series visualizations for request volume and error rates.

### Request Volume Over Time

Track how network request volume changes throughout the day for each mobile application.

```
// Network request volume timeseries by application
fetch user.events, from:-24h
| filter dt.rum.application.type == "mobile" and characteristics.has_request
| makeTimeseries request_count = count(), by:{frontend.name}, interval:1h
```

## Request Volume by Status Code Group

Categorize network requests by HTTP status code group (2xx, 3xx, 4xx, 5xx) to visualize error trends over time. A rising 4xx or 5xx trend may indicate API issues or backend failures.

```
// Network request volume over time by status code group
fetch user.events, from:-24h
| filter dt.rum.application.type == "mobile" and characteristics.has_request
| filter isNotNull(http.response.status_code)
| fieldsAdd status_group = if(http.response.status_code &lt; 300, then:"2xx
Success", else:if(http.response.status_code &lt; 400, then:"3xx Redirect",
else:if(http.response.status_code &lt; 500, then:"4xx Client Error", else:"5xx
Server Error")))
| makeTimeseries request_count = count(), by:{status_group}, interval:1h
```

## Interpreting the Results

| Pattern | What It Means | Action |
|---|---|---|
| **Spike in 5xx errors** | Backend service failures | Check backend service health, recent deployments |
| **Rising 4xx errors** | Client-side issues (bad URLs, auth failures) | Review API contracts, check token expiration logic |
| **Request volume drop** | Users abandoning the app or connectivity issues | Check app crash rates, network availability |
| **Slow upward trend** | Growing user base or increased API chattiness | Plan capacity, consider request batching |

# 7. Performance Optimization Tips

Once you have visibility into your mobile network requests, apply these optimization strategies to improve the user experience.

## Reduce Payload Sizes

| Strategy | Impact |
|---|---|
| **Enable gzip/brotli compression** | Substantially smaller text payloads (JSON compresses well) |
| **Use pagination** | Avoid loading entire datasets at once |
| **Return only needed fields** | Use GraphQL or sparse fieldsets to minimize JSON payloads |
| **Optimize images** | Serve appropriately sized images via CDN with content negotiation |

## Minimize Round Trips

| Strategy | Impact |
|---|---|
| **Batch API calls** | Combine multiple requests into a single endpoint |
| **Use HTTP/2 or HTTP/3** | Multiplexing reduces connection overhead |
| **Implement prefetching** | Load data before the user navigates to a screen |
| **Cache responses** | Use ETags and Cache-Control headers to avoid redundant requests |

## Leverage CDNs

| Strategy | Impact |
|---|---|
| **Static asset delivery** | Serve images, scripts, and configs from edge locations |
| **API acceleration** | Some CDNs offer API caching and edge compute |

| Strategy | Impact |
|---|---|
| **Geographic proximity** | Reduces DNS, TCP, and TLS times for global users |

### Adapt to Connection Type

| Strategy | Impact |
|---|---|
| **Detect connection type** | Use the SDK's connection type data to adjust app behavior |
| **Low-bandwidth mode** | Reduce image quality, defer non-critical requests on slow connections |
| **Offline support** | Queue requests when offline, sync when connectivity returns |
| **Progressive loading** | Load essential content first, enrich progressively |

> **Tip:** Combine Dynatrace mobile monitoring data with your CDN analytics to get a complete picture of content delivery performance from edge to device.

## Summary

In this notebook, we covered:

- **Automatic HTTP capture** across iOS, Android, Flutter, and React Native platforms
- **Connection type detection** and how network conditions affect request performance
- **Request timing** — the phases of an HTTP request as a diagnostic framework, and what a mobile request event actually records
- **Frontend-to-backend correlation** via the `x-dynatrace` header and, from OneAgent for Mobile 8.333, W3C trace context
- **DQL queries** on `user.events` to retrieve, aggregate, and visualize mobile network request data
- **Slow and failed request analysis** using status code grouping and time-series trends
- **Performance optimization tips** covering payload reduction, round-trip minimization, CDN usage, and adaptive behavior

Network requests are the most tangible touchpoint between your mobile app and your backend infrastructure. Monitoring them effectively ensures you can detect, diagnose, and resolve performance issues before they impact your users.

## Next Steps

Continue to **MOBL-08: Session Replay for Mobile** to see how Session Replay reconstructs what the user did before a failed request or a crash.

### Related Notebooks

| Notebook | Topic |
|---|---|
| **MOBL-06** | Crash Reporting & ANR Detection |
| **MOBL-08** | Session Replay for Mobile |
| **SPANS-01** | Distributed Tracing Fundamentals |

## References


- Dynatrace Mobile App Monitoring
- Dynatrace Mobile SDK Documentation
- Distributed tracing (DT docs)
- Dynatrace Query Language (DT docs)


---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*